SN Computer Science (2024) 5:645
https://doi.org/10.1007/s42979-024-02979-y

ORIGINAL RESEARCH

A Concrete Syntax Transformation Approach for Software Language Processing

Kevin Lano[1] · **Qiaomu Xue**[1] · **Howard Haughton**[2]

Received: 17 May 2023 / Accepted: 2 May 2024
© The Author(s) 2024

Abstract
In this paper we present an approach for software language processing tasks such as code generation, reverse-engineering and refactoring, based upon text-to-text (T2T) transformations expressed using the *concrete syntax* of the source and target software languages. The goal of the approach is to provide simpler and more usable techniques for specifying such tasks, compared to model-to-model (M2M) or model-to-text (M2T) transformation approaches. We evaluate the approach on language processing tasks of domain-specific language (DSL) tool support, software abstraction, model transformation, reverse engineering and program translation, and show that it can be effective for these tasks.

Keywords MDE · MDE usability · Language engineering · Concrete syntax transformations

Introduction

Model-driven engineering (MDE) involves the use of models to define the specifications and designs of software systems, using general-purpose modelling languages (GPLs) such as UML or SysML, or specialised *domain-specific modelling languages* (DSLs). Transformations between models and between models and code are usually achieved by means of *model transformation* (MT) languages such as ATL [19], QVT [51] and many others. Model transformations operate on models which conform to specific *metamodels* defining the permitted structure of the models. For example, the UML class diagram metamodel defines the permitted structures of UML class diagrams, including constraints such that the inheritance relation must be acyclic. Model-to-model (M2M) transformations are typically defined using the entity type names and feature names of the source and target metamodels. There are also model-to-text (M2T) transformations which are used to generate implementation code and other types of text from models, EGL [22] is an example language of this kind.

There has been substantial research on MT languages over the last 20 years, however the industrial uptake of these languages remains limited, and we have therefore adopted an alternative approach using text-to-text (T2T) transformations based on language grammars and expressed using source and target language *concrete syntax*, instead of M2M or M2T transformations expressed in terms of language metamodels.

Software Language Processing

In this paper we focus on four software language processing tasks which often occur as part of MDE developments:

1. Definition of specialised domain-specific languages (DSLs) together with supporting tools.
2. Abstraction (reverse-engineering) of software in third-generation programming languages (3GLs) to specification-level models which express the semantics of the software (*semantic models*).
3. Code generation of programming language code from models.
4. Translation from one programming language to another.

Figure 1 summarises our approach for software language processing in the MDE context. To define language-to-

✉ Kevin Lano
kevin.lano@kcl.ac.uk

Qiaomu Xue
qiaomu.1.xue@kcl.ac.uk

Howard Haughton
howard.haughton@gmail.com

[1] Department of Informatics, King's College London, London, UK

[2] Holistic Risk Solutions Ltd, Croydon, UK

Published online: 13 June 2024

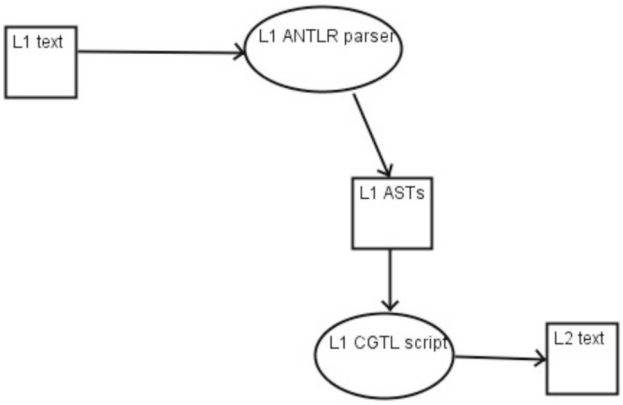


Fig. 1 ANTLR/CGTL language processing process

language transformations we use mappings written in the concrete syntax of the source and target languages. Thus to map Object Constraint Language (OCL) [50] expressions to Java we could have rules such as:

```
_1 mod _2 |-->_1 % _2
_1 div _2 |-->((int) (_1/_2))
```

with the left side of each rule written in schematic concrete syntax of the source language (in this case, OCL) and the right side written in schematic concrete syntax of the target language (Java). Such rules form the basis of a DSL for language processing tasks termed *Concrete Grammar Transformation Language* (CGTL). This is a text-to-text transformation language based on the *Concrete Syntax Transformation Language* (CSTL) code generator language of [41] but generalised to process the parse trees of any software language.

To build grammars and parsers for software languages we use ANTLR [7]. This is a lightweight tool in wide use for language engineering tasks. The use of ANTLR was recommended by [14] in the case of MDE contexts where a lightweight language processing solution is needed. However, other parsing technologies such as SableCC or JavaCC could alternatively be used with our approach.

MDE Usability Issues

MDE has become an established software engineering approach for high-integrity systems such as automotive [21] and aerospace software systems [17, 49]. In such domains, the high quality assurance requirements on software motivates the use of rigorous approaches such as MDE. However, empirical research into MDE use has found that there are significant obstacles to more general use of MDE techniques and tools by software practitioners [5, 23, 27, 52, 58].

These barriers particularly concern:

- The complexity of MDE notations, tools and methods, with corresponding high training and adoption costs.
- Poor tool support for MDE, in particular poor user experience and lack of flexibility and configurability for MDE tools.
- Poor integration of MDE with other development practices such as continuous deployment and DevOps.

These factors impair MDE usability, where usability is understood as the avoidance of excessive effort in tasks, such as when constructing or using MDE software tools.

The effort required to understand and use a large GPL such as UML has been identified as an obstacle to MDE by [1, 5]. An alternative is to use a domain-specific language (DSL), which is a customised modelling language providing a restricted modelling notation for a specific problem domain. A DSL for a given domain enables the domain practitioners to model systems using concepts and notations which are already familiar to these users. DSLs have been widely used to specify specialised software functionalities or categories of software applications such as machine learning systems [63] and text processing [18]. Use of a DSL to define models reduces (in principle) the effort required to develop tools to analyse models and to generate code from models, compared to the use of more general-purpose modelling notations such as UML and OCL. However, DSLs need careful design to ensure that their constructs are appropriate for the intended domain, and can be used by the intended end users. The definition of a DSL often needs to be extended or modified over time, e.g., to add new features or constructs. DSL-specific tools need to be developed, and we consider this issue in Sect. "DSL Definition and Tool Support Using ANTLR/CGTL".

Particular problems have also been identified with model transformation languages and tools [26, 31]. These include:

- Lack of transformation repositories and transformation reuse capabilities. These derive in part from tight couplings between specific transformations and specific metamodels.
- Complexity of the supporting ecosystem. This is the number of different tools that are needed to support the creation, management and transformation of models.
- Steep learning curves. The MT languages and their supporting ecosystem concepts and technologies require significant effort to learn.
- Lack of tool maturity. Tools have many bugs and flaws impeding their use.

The papers [12] and [26] consider the current usage and future prospects for specialised model transformation lan-

guages, and identify that many practitioners prefer to use general-purpose programming languages such as Java to write transformations, due to the relatively poor knowledge and tool support available for MT languages.

Proposals to address the usability problems of MDE include the use of hybrid methods that combine agile development and MDE [4], and the use of simplified transformation languages and MDE processes [14, 24, 28]. Here, we will particularly follow the latter approach.

Research Contributions

The contributions of the research described here are:

1. Based on our experience with the industrial application of MDE in finance [42], we confirm the insights of [14, 24] and [28] regarding the usability benefits of grammar-based solutions and concrete syntax specifications for language processing MDE tasks.
2. We provide a text-to-text transformation language for language processing tasks, CGTL, together with an implementation of CGTL in the AgileUML toolset [20].
3. We enable the reuse of existing grammars and parsers, such as ANTLR grammars and derived parsers.
4. We demonstrate that the approach can be applied to diverse tasks such as DSL tool support, refactoring, and program translation.
5. We demonstrate that the approach is practical to use, both in terms of the effort required to develop CGTL solutions, and in terms of the computational efficiency of these solutions.

The paper is structured as follows: Sect. "CGTL and CSTL" introduces CGTL. Applications of the ANTLR/CGTL approach to several diverse case studies of DSL definition, programming language abstraction/translation, and model transformations are described in Sects. "DSL Definition and Tool Support Using ANTLR/CGTL", "Software Language Translation" and "Model Transformations". Section "Evaluation and Comparison" evaluates the outcomes of the case studies, Sect. "Related Work" compares our approach with other related works, Sect. "Limitations" discusses limitations, and Sect. "Conclusions and Future Work" gives the conclusions and future work.

CGTL and CSTL

In this section we describe the syntax and semantics of CGTL and compare it to other concrete syntax transformation languages such as CGT [24] and Gra2MoL [28].

CSTL and CGTL Syntax and Semantics

The definition of CSTL and CGTL scripts is organised into *rulesets*, which have the syntax

```
rulesetName::
rule1
...
ruleN
```

Each source language syntactic category has a corresponding ruleset which processes source elements in that category. Individual rules consist of a left hand side (LHS) which selects (by pattern matching) certain source elements in the category, and a right hand side (RHS) that describes the result of applying the rule to produce a corresponding target element for each selected source element. The LHS uses source language concrete syntax for input elements, possibly with metavariables *_i* for $i \in 1..99$ representing subelements, and the RHS expresses in target language syntax the corresponding output element for each input element. In the RHS each *_i* denotes the translation of the source element bound to *_i* on the LHS. Rule *conditions* testing properties of the input elements can also be included after the RHS text, together with rule *actions* setting such properties, so that the general CSTL syntax of a rule is:

```
LHS |-->RHS[<when> conditions]
[<action> actions]
```

The syntax of a basic condition or action is a pair of a metavariable and a *property* identifier, where the property can denote a type (such as *Map* for map-typed elements) or a syntactic class (such as *enumerationLiteral* to indicate that the item is an element of an enumerated type):

```
_i property
```

The *conditions* and *actions* are comma-separated sequences of such clauses:

```
metavariable property
[, metavariable property]*
```

The LHS of a rule cannot be empty, but the RHS can be (in which case empty text is produced by the rule).

Execution of CSTL scripts is based on pattern matching of source elements with the LHS of rules, and text substitution of mapped target elements into the RHS of rules. Given an

input element in the source category of the ruleset, successive rules are tested for a match with the element. The first matching rule whose conditions are all true is then applied to the element. The actions of the rule are also executed.

Apart from rulesets for source language syntax categories, custom functions can also be defined as rulesets within a script. These functions can be used to inspect input elements to any depth, or to compute results based on the input elements. Such rulesets f are explicitly invoked on elements by the syntax `_i`f` in the RHS, condition or actions of a rule.

CGTL retains the same syntax and concepts as CSTL but internally it processes parse trees or abstract syntax trees (ASTs) instead of UML/OCL model data. It can be applied in principle to any source language that has a formal grammar.

To illustrate the use of conditions and actions in CGTL, the abstraction of Pascal to UML/OCL involves processing of Pascal variable declarations of the form `var x: T` to translate them into corresponding UML/OCL declarations. This is achieved by CGTL rules of the form:

```
var _1 : integer |-->  attribute _1 :
 int := 0;<action> _1 int
var _1 : real |-->  attribute _1 :
double := 0.0;<action> _1 double
var _1 : _2 |-->  attribute _1 : _2
:= Set{};<when> _2 Set <action> _1 Set
```

The first two rules translate Pascal variable declarations to UML attribute declarations and assert that the declared identifiers bound to _1 have the (OCL) types corresponding to their Pascal types. The final rule matches against variable declarations whose type is a Pascal `Set` type and asserts that the variable has an OCL Set type. The asserted typing information for the declared identifiers bound to _1 in the above rules can then be tested in the conditions of subsequently executed rules.

CGTL incorporates several extensions to CSTL: (i) pattern-matching of a single metavariable against multiple subchildren of a parse tree; (ii) recursive application of a ruleset; (iii) global script variables; (iv) nested rule/script invocation in rule conditions and actions, and invocation of external (Java) functions; and (v) dynamic loading of scripts.

The metamodel of AST terms which we use to represent parse trees is shown in Fig. 2. In it, the $features$ map of $ASTTerm$ records associated information about the language elements represented by the terms, such as their type, stereotypes or tagged values. This information can be set by CGTL actions and read by CGTL conditions. For example, if a variable with name “x" has been recognised as having $double$ type, $ASTTerm.features$[“x"] would include the string “double".

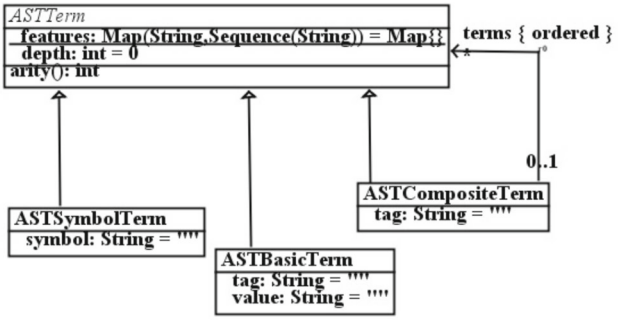


Fig. 2 Metamodel of parse trees

Rule condition properties in CGTL can be the names of source grammar non-terminal symbols tg. Such conditions `_i tg` restrict matches of the rule to terms whose i’th subterm (an $ASTTerm$ instance) has this tag.

We write composite syntax tree terms with tag tg and n subterms as LISP-style lists $(tg\ t_1 ... t_n)$. Basic terms with tag tg and value v are written as $(tg\ v)$. Symbol terms represent literal terminal symbols in the grammar, such as ‘+’ or ‘mod’.

CGTL was designed to be simple to understand and intuitive to use for general software practitioners. An insight from the survey of [26] is that the concept of a *mapping* from one language or metamodel to another is a key means by which users understand and construct model transformations. Likewise, they point out the benefits for expressiveness and productivity of MTL pattern-matching facilities. CGTL is based on both principles.

Comparison with Other Concrete Syntax Transformation Languages

Concrete syntax-based transformation languages have been proposed by [8, 24, 28]. The *Concrete syntax-based graph transformation* (CGT) language of [24] enables the concrete graphical syntax of graphical modelling languages to be directly used to specify transformation rules. The authors of [24] find that CGT is substantially more concise than metamodel-based MT languages when applied to the same problems. The approach of [8] uses a graph grammar based MT language to specify rules for transformations on graphical languages using concrete syntax, however these rules need also to refer to the names of metamodel entities and features in general. Graphical concrete syntax transformation languages require specialised tool support for visual rule specifications, and the uptake of such transformation languages has been limited. We prefer to focus on text-based specification languages, which only depend upon relatively simple text editors to define transformations.

The Gra2MoL text-to-model language of [28] provides grammar-based facilities to search and extract information from syntax trees, with benefits in terms of conciseness and

clarity compared to conventional MT languages. However these facilities are more complex to code, and are more closely bound to a particular source grammar, compared to the pattern-matching facilities of CGTL. For example, to process Pascal-style assignment statements, the query

```
from statement{TOKEN[0].eq(":=")} st
```

is used in Gra2MoL to specify the source element *st* to be processed [28]. The subparts of *st* are then referred to using explicit grammar non-terminal symbol names (*designator* and *expression*):

```
lElem : /st/#designator;
rElem : /st/#expression;
```

In contrast, a corresponding CGTL rule f to process assignment statements would have the form:

```
f::
_1 := _2 |-->RHS
```

Provided that the concrete syntax is sufficient to unambiguously match assignment statements, there is no need to refer to the grammar non-terminal symbol names. As with Gra2MoL, CGTL can also define queries to navigate deeply into parse tree terms, using called functions g as `_i`g to test or extract information from a term bound to $_i$. This approach is heavily used in the mobile app DSL (Sect. "Mobile App Specification"). Otherwise, Gra2MoL involves the detailed creation of target models in a similar manner to ATL rules, and hence requires knowledge of the target metamodel.

Model-to-text transformation languages such as EGL [22] and Xtend [59] use concrete syntax templates for target language production, however the source models are navigated via metamodel features. In contrast, CGTL uses concrete syntax templates for both source and target languages.

DSL Definition and Tool Support Using ANTLR/CGTL

DSLs can be specified using metamodels, but another effective means for defining a textual DSL is by means of a language grammar. In our approach for language processing we use ANTLR to define a grammar for a textual DSL, and to generate parsers for the DSL. Tools to process DSL models (expressed as texts conforming to the DSL grammar) then operate on the DSL syntax trees produced by the DSL parser to produce text in a target language. The DSL tools are written using CGTL. CGTL scripts can produce output text in a wide range of formats, including HTML, XML and programming language code.

The typical design of a DSL tool using the ANTLR and CGTL approach uses the toolchain of Fig. 1 with $L1$ being the DSL, and $L2$ being the target language to produce, such as English in the case that the tool produces a natural language textual explanation of the DSL specification, or a programming language in the case of producing executable code implementing the specification. To assist in the construction of DSL tools, we provide a command line tool *antlr2cstl* to automatically produce an outline CGTL script from an ANTLR grammar file $LParser.g4$ for a language L. The resulting script contains the appropriate rulesets and rule left hand sides and rule conditions to process trees output by $LParser.g4$. Each possible grammar production of an ANTLR grammar rule r results in a separate template rule in the CGTL ruleset for r. The DSL tool designer then needs to complete the rule definitions to achieve the required transformation.

In the following subsections we describe how tools for three example DSLs have been developed using ANTLR/CGTL.

Natural Language Processing for Requirements Formalisation

The goal of this project was to produce a DSL capable of expressing natural language processing pipelines in a high-level notation supported by tools, including a code generator targetting Python NLTK.[1]

Natural language processing (NLP) is a key technique for many applications which process or produce natural language text or speech. In particular, NLP has been used to formalise software requirements expressed in natural language documents [13, 57, 60, 61]. Within a collaborative project with Sheffield University [54], we identified processing steps, actions and data which are widely used in NLP for requirements formalisation, and we codified these as a simple DSL with a syntax based on SQL. SQL is used because of its declarative nature and relative simplicity, facilitating use by NLP practitioners who may not be familiar with programming languages.

A CGTL script *nlp.cstl* was defined to translate DSL specifications to Python NLTK code. The DSL has been tested with a wide range of NLP pipelines including POS-tagging, chunking, entity extraction and the derivation of UML use cases from informal user stories (Sect. DSL Definition and Tool Support).

[1] www.nltk.org.

Mobile App Specification

The goal of this DSL is to enable mobile apps to be defined in a platform-independent manner via specifications of the business data and services supported by the app. DSL tools are needed to generate app code in one or more mobile platforms such as Android or iOS. The iOS SwiftUI platform was selected as the priority target.

App specifications in the DSL consist of:

- Textual UML class definitions of business entities, containing attribute definitions, and annotated with UML stereotypes identifying the entity data storage location (remote or local) and persistence (data is retained persistently or not).
- Textual use case definitions specifying the offered services of the app.

We developed a grammar *Mobile.g4* for the DSL, and DSL code-generation tools specified in CGTL scripts. The generated app architecture follows the Model-View-ViewModel (MVVM) style. The DSL use case definitions are mapped to operations of a business tier facade component, and public use cases are also used to produce SwiftUI views (screens) that trigger the business tier operations. Value objects are used to transfer business data between tiers.

The CGTL scripts are organised on the basis of the different mobile app target files which are generated: each script produces a specific component within the app architecture, such as a model facade, value object class, database interface or UI screen. The DSL and tools were tested with a wide range of mobile app specifications including finance and health apps (Sect. DSL Definition and Tool Support).

Financial Model Specification and Algebraic Simplification

This project was a collaboration between a financial services company, King's College London and a MDE tool provider. Its goal was the development of a DSL and tools to support financial model specification, validation and implementation [25]. Mathematical financial models define different financial products and their theoretical properties. An example of such a financial model is the Black-Scholes theory for option pricing, based on a probabilistic theory of underlying asset prices [9]. Mathematical financial models typically use classical mathematical concepts such as integration, $\int_a^b$, differentiation, ∂_x, statistical expectation $E[expr]$, and standard functions such as exponentiation e^x, combinatorial C_k^n and square root $\sqrt{x}$.

The project defined the *MathOCL* DSL [25] to enable finance practitioners to express and manipulate financial models, and this DSL supports the necessary mathematical concepts. It also provides the following constructs:

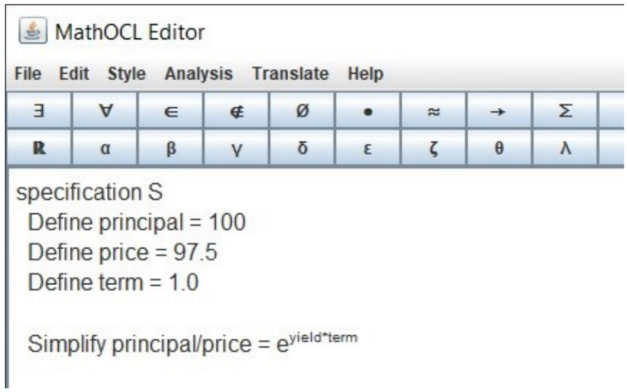


Fig. 3 MathApp tool and example model

- Constraint on $var \vdash expr$
  Assert that $expr$ holds for variable var for the purpose of the model, e.g., that a variable is positive.
- Define $var = expr$
  Define a variable or function by an expression, which can also involve instructions such as $Factor\ expr\ by\ v$, $Cancel\ v\ in\ expr$, etc.
- Define $var \sim D$
  Define var as a random variable following probability distribution D.
- Solve $eqns$ for $vars$
  Solve one or more equations for a variable/list of variables. Quadratic, differential and multiple linear equations can be solved (symbolically).
- Simplify $expr$
  Apply algebraic simplification to reduce $expr$ to a simpler form.

The DSL was designed by the finance collaborator, a domain expert, and the first author (as a language engineer) working together. An ANTLR grammar MathOCL.g4 and its corresponding DSL tools were developed. A graphical editor, MathApp, was defined for the input and manipulation of MathOCL specifications (Fig. 3). This tool converts the graphical mathematics notation such as e^x into plain text equivalents:

```
e^{x}
```

and then applies a CGTL script *simplify.cstl* to perform algebraic simplification and evaluation on the resulting MathOCL text (the lower panel of Fig. 4 shows the text model after algebraic simplification).

In contrast to the tools for the NLP and mobile app DSLs, which use the process of Fig. 1, for MathOCL a more sophisticated process is used for tool support using

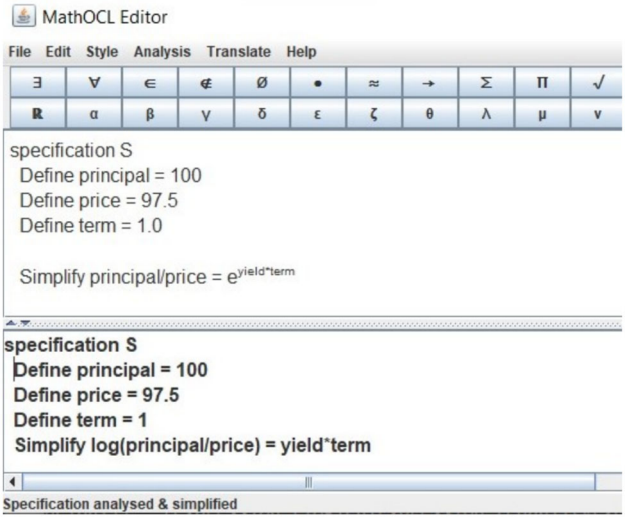


Fig. 4 Algebraic simplification of example model

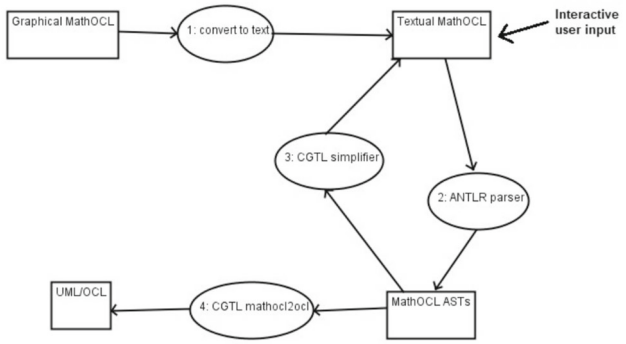


Fig. 5 ANTLR/CGTL tool process for MathOCL

ANTLR and CGTL, as shown in Fig. 5. This process translates (in step 1) the graphical MathOCL specification to a plain text MathOCL document, which is retained for the duration of a MathApp tool session. The document can be operated on by any CGTL script for the MathOCL language. In particular, this process supports *recurrent* application of the *simplify.cstl* script, to perform repeated simplification/reduction of the DSL specification until no change is observed (steps 2 and 3 of Fig. 5). It also enables *interactive* use of the tools, because the user can edit the MathOCL text document at any point in the processing, in order to add instructions for specific types of algebraic manipulation. Translations to other notations are also supported. A translation script *mathocl2ocl.cstl* from MathOCL to standard OCL is provided (step 4) in order to provide a bridge to the AgileUML code generators, for example, to produce C# or Python code from a financial model. The algebraic simplifier *simplify.cstl* and other CGTL scripts can also be applied to MathOCL texts via the command line, as an alternative to the graphical MathApp tool interface.

Section "DSL Definition and Tool Support" evaluates the effectiveness of the MathOCL DSL and tools.

Software Language Translation

Translation between programming languages (sometimes referred to as *transpilation*) has become a significant area of research due to industrial needs to migrate and modernise legacy software [2, 32, 34, 47]. Critical software systems exist in antiquated languages such as COBOL or old versions of Visual Basic (VB), and need to be translated to modern programming languages in order to be supported and maintained. The costs and time required for manual program translation can be prodigious. For example, migration of 35 million lines of a critical banking system written in Python 2.7 to Python 3 took over 3 years of effort at JP Morgan [55]. Automated program translation is therefore an attractive alternative to manual translation or redevelopment of software assets. In addition to source-to-target translation, it is also useful to perform *reverse engineering* to obtain design or specification representations from a software system's code. This facilitates code comprehension, quality analysis and quality improvement through processes such as refactoring [36].

Existing transpilers such as [29, 32] have significant limitations, especially when translating between languages of different categories such as Python (implicitly typed, prototype-based) and Java [47] (explicitly typed and object-oriented). Machine learning approaches such as [32] construct an implicit semantic understanding of source code based on training examples, but this understanding can fail to be correct in complex cases, e.g., to translate general format Java *for* loops into Python, or to correctly translate Java arrays to Python lists [47]. On the other hand, explicit encoding of translations in a manually-programmed tool such as [29] involves substantial effort and can suffer from incompleteness.

Instead, we use the ANTLR/CGTL approach to define abstraction transformations which incorporate precise and explicit semantic modelling of programs in Java, C, Visual Basic 6 (VB6), COBOL '85, Python, Pascal and JavaScript.

To translate programs of source language $L1$ to programs of target language $L2$, we use an ANTLR parser for $L1$ to produce $L1$ parse trees (step 1 of Fig. 6), which are then input to an abstraction transformation for $L1$, written using CGTL (step 2 of Fig. 6). The output is a UML/OCL specification in textual form, consisting of class specifications with data features and operations, and use cases defining global processing. The specification may utilise the operations of OCL libraries for files, dates, etc. [39]. The specification forms a semantic model of the source system, which is intended to express the complete semantics of the source code in a rel-

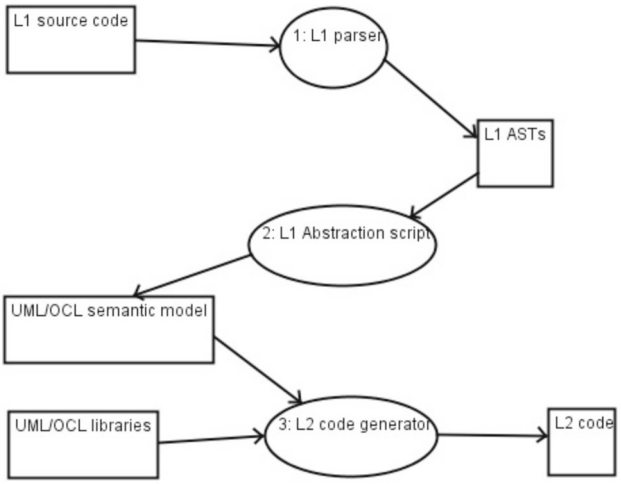


Fig. 6 Program translation process

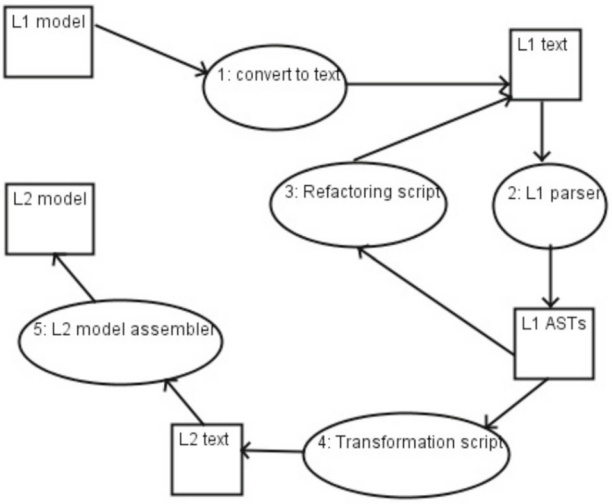


Fig. 7 Model transformation process using CGTL

atively high-level form (in contrast to compiler intermediate representations such as LLVM [45]). Forward engineering using CSTL/CGTL code generators or other MDE code generation techniques can then be employed to map the abstracted specification to the target programming language (step 3 of Fig. 6).

In Sect. "Program Abstraction and Translation" we evaluate the effectiveness of the ANTLR/CGTL approach for program translation.

Model Transformations

General model transformations (MT) are model-to-model transformations which map models of one language $L1$ to models of another, $L2$, or which refactor (update in-place) a model of a single language. The first kind of transformation is termed *exogenous*, and the second *endogenous*.

Although CGTL was not intended for general MT tasks, it can be used for these by:

1. For an exogenous transformation, processing a $L1$ textual representation of the source model using CGTL rules to produce target language $L2$ text (steps 1, 2 and 4 in Fig. 7), and then building an $L2$ model using a parser/model assembler which creates a target model from target language text (step 5).
2. For an endogenous transformation, iteratively re-applying the transformation on the $L1$ (= $L2$) text (steps 2 and 3) until a required result is obtained, prior to model assembly (step 5 for $L1$).

The advantage of this scheme is that the CGTL rules (of steps 3 or 4) express the key idea of the transformation using the concrete syntax of $L1$ and $L2$, abstracting from the details of navigating (step 1) and assembling (step 5) models. Because of step 1 in Fig. 7, the CGTL rules are independent of the $L1$ metamodel. Because of step 5 they are also independent of the $L2$ metamodel. The task of ensuring consistent target model structure is delegated to the model assembler. The disadvantage is the additional processing costs of parsing and target model assembly.

We compare the ANTLR/CGTL approach for defining model transformations to other approaches, on specific transformation tasks, in Sect. "Model Transformation Specification".

Evaluation and Comparison

In this section we evaluate the effectiveness of the ANTLR/CGTL approach for a range of software language processing tasks. We evaluate the effectiveness of the tools constructed using CGTL in terms of their achievement of the task goals, their performance, and the effort expended to construct and extend the tools. We also compare the ANTLR/CGTL approach with alternative approaches, with regard to the effort expended and the knowledge required to create and extend the transformation artefacts.

All execution times are measured on a Windows 10 quad-core 64-bit laptop (Intel i5-7440HQ 2.80GHz processor) with 8GB RAM and 25% processor allocation. Times are computed using Java *Date/time* as the average of 3 independent executions, with no other processes consuming significant resources on the machine. All artefacts used in this evaluation are provided at zenodo.org/records/10039613.

Table 1 CGTL performance on NLP cases

Case	Model size (LOC)	Code generation time
NLP Pipeline 1	12	0.3s
NLP Pipeline 2	17	0.4s
Use case derivation	22	0.3s
Entity extraction	22	0.3s
NLP Pipeline 5	24	0.5s
NLP Pipeline 6	24	0.83s
NLP Pipeline 7	12	0.85s
Regex matching	10	0.1s
Fuzzy similarity	12	0.7s
Averages	17.3	0.48s

DSL Definition and Tool Support

As described in Sect. "DSL Definition and Tool Support Using ANTLR/CGTL", we applied the ANTLR/CGTL approach to define three DSLs and their support tools:

1. For natural language processing (NLP) as part of a requirements formalisation process (Sect. Natural Language Processing for Requirements Formalisation).
2. For mobile app specification (Sect. Mobile App Specification).
3. For financial model specification (Sect. Financial Model Specification and Algebraic Simplification).

Here we evaluate the outcomes of these developments by applying the DSL tools to a range of specification examples in each DSL.

Experimental Design

The DSL tools were applied to example DSL specifications, in the case of the NLP DSL these examples were selected from case studies being used by the requirements formalisation project. For the mobile DSL the examples were selected by the DSL developer. In the case of MathOCL, the examples were based on financial model analysis cases provided by the industrial collaborator. The cases are given in the *nlpDSL*, *mobileDSL* and *mathOCL* directories of the Zenodo repository.

For each example, we measure the size of the generated code and the execution time of the CGTL script. We also measure the CGTL script size for each DSL, and estimate the development effort used to create the script. In the case of the mobile app DSL, we also compare the development effort and complexity to the tools produced by the AppCraft project [6], which developed a closely-related mobile app DSL and tools using Xtext [59] and the Xtend model-to-text language. The AppCraft DSL has the same functional and data specification notations as our Mobile language, but also adds notations for UI design (in our DSL approach the UI designs are generated from the data and functions).

Experiment Results

Regarding the efficiency of the produced DSL tools, Table 1 shows the Python code generation time for the 9 test cases of NLP DSL models. The generated code size is 4KB in each case.

Table 2 gives the model size and code generation time for the 10 mobile app DSL test cases.

Table 3 gives the model size, generated code size (in Java or C#) and analysis time (Step 3 in Fig. 5) for the 7 MathOCL DSL test cases.

Regarding the effort expended in development of the tools, Table 4 summarises the size of the DSL tool scripts and the effort required to construct them.

Table 5 gives the comparison of the ANTLR/CGTL mobile DSL tooling compared to the AppCraft Xtext tooling.

Discussion

In each DSL case, the goals of the DSL development were achieved by means of using the ANTLR/CGTL approach for DSL definition and tool support.

Overall, the data of Tables 1, 2, 3 demonstrate that the DSL tools defined using CGTL are able to perform DSL transformations upon representative DSL specifications in a practical time. Because a CGTL script processes each node of a parse tree once for each rule which directly refers to the node, the time complexity of processing is generally linear in the input parse tree size (number of nodes in the tree).

The data of Table 4 means that on average 114 LOC of CGTL can be written each day for these DSL tools, and this level of productivity enabled the successful completion of the projects on time.

Table 2 CGTL performance on mobile app cases

Case	Model size (LOC)	Generated code size	Code generation time
Person app 1	32	28KB	0.8s
Student app	60	39KB	1.47s
Person app 2	40	29KB	1 s
Person app 3	46	30KB	1.2s
BMI app	15	14KB	1.1s
Health app	40	22KB	1.96s
Bond app	120	23KB	2.81s
Person DB app	34	32KB	2.3s
Student DB app	60	60KB	4.0s
Student app cloud + DB	60	50KB	1.8s
Averages	50.7	32.7KB	1.84s

Table 3 CGTL performance on MathOCL cases

Case	Model size (LOC)	Generated code size	Analysis time
Binomial model	17	362 LOC	0.2s
Replicating portfolio	15	320 LOC	0.3s
BS Quadratic	11	255 LOC	0.25s
Proof example	4	–	0.14s
Statistics example	20	–	0.43s
Thiele's equations	15	145 LOC	0.89s
Bond valuation	16	297 LOC	0.44s
Averages	14	275.8 LOC	0.38s

Table 4 Script size and development effort for DSL tools developed using CGTL

Script	Size (LOC)	Effort (person days)
nlp.cstl	112	2
Mobile app scripts	1870	10
simplify.cstl	1919	22
mathocl2ocl.cstl	1023	11
Averages	1231	10.8

In comparison to the 10 person days required for the ANTLR/CGTL mobile DSL construction, the AppCraft DSL took over 2 person months to develop (Table 5), due to the high complexity involved in writing correct Xtext grammars, which combine parsing and semantic analysis. From a DSL grammar, Xtext derives a metamodel for the DSL, together with a DSL parser, however this dual interpretation of an Xtext grammar has deficiencies in terms both of the quality of the metamodel (i.e., the metamodel is at a low level of abstraction) and restricting the reusability of the grammar (i.e., to reuse the grammar for a different parser generator such as ANTLR requires the removal of all the metamodel-construction elements from the Xtext grammar) [28]. Using Xtext for this task involves writing code in 4 different languages (Ecore, Xtext, Xtend and Java) and managing large numbers of generated artefacts.

Overall we can conclude that the combination of ANTLR and CGTL has been shown to be effective for defining text-

Table 5 Comparison of mobile DSL tooling approaches

	DSL grammar size (LOC)	Tool size (LOC)	Source artefacts/languages	Development effort (person days)
ANTLR/CGTL	274	1870	16/2	10
Xtext/Xtend	543	5379	17/4	44

based tools for DSL processing, since the effort required for writing grammars and CGTL scripts is relatively low, and the efficiency of script execution is adequate for practical use. The expertise required to define CGTL scripts is (i) understanding of the source language syntax and semantics; (ii) understanding of the target syntax and semantics; and (iii) knowledge of CGTL. There is no need to understand the metamodels of the source or target languages.

Program Abstraction and Translation

In this section we evaluate the ANTLR/CGTL approach for program abstraction and translation, described in Sect. 4 above.

Experimental Design

To investigate the application of ANTLR/CGTL for program abstraction and translation we developed and evaluated example abstraction translations from Java 6/7, JavaScript, Python, Pascal, C, COBOL '85 and Visual Basic 6 to UML/OCL. These were then applied to over 600 examples, including real-world programs from finance applications. The principal criteria for selection of examples was to achieve complete or high coverage of the grammars for each language. Coverage of core libraries was also an aim (e.g., for Java, the java.io, java.lang, java.util and java.math libraries, Excel worksheet functions for VB, and for Python the datetime, re and math libraries, and partial coverage of numpy). In addition, actual legacy programs were selected (a legacy bond analysis application in the case of VB [36], and the legacy billing application of [40] for COBOL).

The source code of program translation cases is contained in the *sources*, *vb6*, *javascriptsources*, *python Abstraction*, *pascalAbstraction*, *csources* and *cobol* directories of the Zenodo repository.

Forward engineering was applied to the abstracted UML/OCL models to generate code in selected target languages (e.g., Java and C# in the case of Python source programs). To evaluate the correctness of the program translations, we use the *computational accuracy* measure of [32]. This measures the percentage of tests out of a set of test cases for a program that return the same results when run on the original source program and on the translated target. The same test values/parameters are used for both source and target programs. This is an appropriate measure because it indicates to what extent the translated target preserves the semantics of the source program.

Tests were prepared for each program case based on the documentation for the example source programs, such as the language manuals [16, 48], the books [30, 53] and the python.org Python language reference.

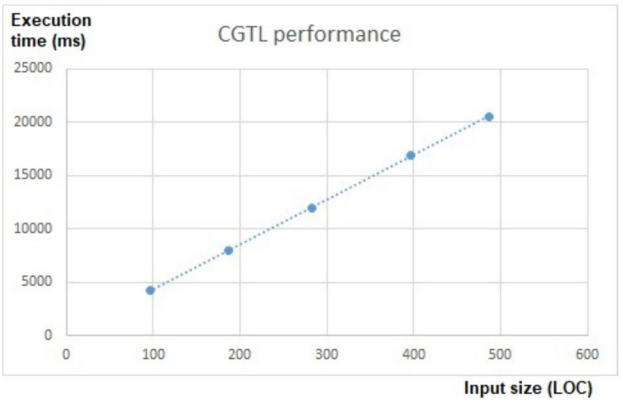


Fig. 8 Abstraction times for Python source programs

We also investigated the efficiency and time complexity of the translators by selecting or creating source code examples across a range of sizes and analysing the execution time, using the same experimental settup as for the DSL tool analysis.

Finally, we estimate the development effort required to construct the program translation scripts in CGTL, and the effort required to make changes of differing scale to the scripts. These figures are compared to the development/change effort required for previous versions of the UML to Java, Java to UML, C to UML and JavaScript to UML tools which were coded in Java. These previous tool versions have similar functionality to the ANTLR/CGTL versions.

Experiment Results

The computational accuracy results calculated over all programs in each translation direction are shown in Table 6.

In terms of the efficiency of the translation process, the abstraction scripts have linear time complexity. Figure 8 shows the time in ms (y-axis) for abstraction of Python programs of different sizes, shown in LOC on the x-axis. Similar performance results hold for abstraction of other source languages.

Columns 2 and 3 of Table 7 summarise the script size and estimated effort (in person months) required to construct CSTL/CGTL abstraction and code generation scripts. The code generation scripts are written in CSTL, whilst the abstraction scripts are written in CGTL. Columns 4 and 5 give the corresponding data for Java-coded versions of the UML to Java, Java to UML, C to UML and JavaScript to UML tools. The UML/OCL to Java code generator was written in Java over the period 2003 to 2023 and comprises over 20,000 LOC, with an estimated total resource consumption of 3 person years. Similar figures are given for the Java-coded UML/OCL to C++ code generator in [35]. Initial work on program abstraction also used Java-coded processing of

Table 6 Evaluation cases for program abstraction/translation: computational accuracy

Target language	Source language						
	Java 6/7	C	JS	VB6	COBOL	Python	Pascal
Python	93%	–	95%	84%	–	–	–
Swift	96%	84%	–	–	–	–	–
C#	96%	90%	–	–	92%	91%	–
Go	90%	91%	–	–	–	–	–
Java 8	98%	–	–	–	87%	83%	91%
C++	93%	–	–	–	–	–	–
C	86%	–	–	–	–	–	–
Averages	93%	88%	95%	84%	89%	88%	91%

parse trees to build UML/OCL specifications as instances of the AgileUML UML/OCL metamodel [33]. These also consumed about 3 person years of effort in total. All work in Table 7 was performed by the first author using similar work practices.

Table 8 compares the development effort required to make specific changes in the CGTL and Java-coded abstractors.

Discussion

The results of Table 6 for translation accuracy are generally higher for translation between similar languages such as C# and Java, and lower for more distantly-related languages such as VB6 and Python. Java to Python translation is more accurate than the reverse direction, because of the lack of explicit typing in Python. The same pattern for semantic accuracy results has been observed in other program translation work [3].

Regarding the time complexity of translation, this was found to be linear in terms of the lines of source code (Fig. 8). It could be expected that processing of a parse tree of depth n will require at least 2^n steps, because there are $2^n - 1$ nodes in a complete binary tree of depth n, however the trees which arise from the parsing of programs tend to have relatively shallow depth compared to their breadth—because they represent sequences of program elements such as declarations and statements, and so the parse tree size and time complexity for processing is determined by the breadth factor, i.e., the LOC.

Table 7 shows that the average productivity for writing CSTL/CGTL for code generators and abstractors is around 536 LOC per person month, a less productive result than for DSL tools, which is perhaps due to the highly complex nature of the source languages in contrast to DSLs. In comparison to the CGTL productivity figure, the 4 Java-based code generators and abstractors involved an average productivity of 648 LOC per person month.

However the total effort required to develop CSTL/CGTL scripts for these tasks is substantially lower than that required to develop code generators or abstractors using Java: this total is an average of 3.4pm for the four CGTL scripts with Java equivalents, versus an average 18pm for the Java versions—a factor of 5 reduction in effort. The reason for this difference is due to the need in the Java versions to navigate the UML/OCL metamodel, and (for Java-coded abstractors) to navigate programmatically through parse trees with explicit testing of tree tags and arities. In contrast, CSTL/CGTL scripts abstract from the details of metamodels and parse tree structures, so that the writer of a script needs only to express the transformation involved in terms of the concrete syntax of the language elements involved, either for abstraction or code generation. A similar reduction in effort through the use of CGTL is observed in Table 8.

Model Transformation Specification

In this section we evaluate the effectiveness of the ANTLR/CGTL approach for model transformation definition (Sect. Model Transformations), and compare it with M2M approaches using transformations coded in Java or in a MT language.

Experimental Design

We analyse four CGTL-coded transformations devised for the MathOCL language: *simplify.cstl*, *mathocl2ocl.cstl*, *mathocl2matlab.cstl*, *mathocl2mamba.cstl*, together with the *restructure.cstl* transformation for code refactoring. We estimate the size and development effort for these transformations, and we compare these figures to the size and effort for similar model transformations which were coded using Java or UML-RSDS [33] and that form part of the AgileUML tools. These transformations consist of a UML2C code generator developed using UML-RSDS [44], and translations from ATL, ETL and QVTr to UML-RSDS, coded using Java [33, 37]. The source code of all these transformations may be found in the *transformations* directory of the Zenodo repository.

Table 7 CSTL/CGTL script size and development effort compared to Java MTs

CGTL Script	Script LOC	Script effort	Java MT (LOC)	Java MT effort
UML2Swift5	723	3.5pm	–	–
UML2Java8	562	3pm	20,000+	36pm
UML2Go	1142	2.3pm	–	–
Java2UML	3292	5.5pm	12,000	18pm
C2UML	2160	2.5pm	8,000	10pm
JS2UML	953	2.5pm	6,000	7pm
VB2UML	2430	4.5pm	–	–
COBOL2UML	3818	6pm	–	–
Python2UML	1705	2.7pm	–	–
Pascal2UML	1175	1pm	–	–
Averages	1796	3.4pm	11,500	18pm

Table 8 Comparison of abstractor enhancement change costs

Change	Effort for CGTL abstractor	Effort for Java-coded abstractor
Java: Add abstraction	2 min	30 min
of System.arraycopy(u,i,v,j,k)	1 LOC	67 LOC
JavaScript: add generator	15 min	60 min
functions & yield statement	7 LOC	86 LOC
C: define complete		
mapping from C *declarator*	3 h	7 h
to UML/OCL declarations	230 LOC	450 LOC

Table 9 Size and development effort for CGTL-coded model transformations

Script	Grammar size		Transformation	Size	Effort
	Source	Target	kind	(LOC)	(person days)
simplify.cstl	134	134	Endogenous	1919	22
mathocl2ocl.cstl	134	173	Exogenous	1023	11
mathocl2matlab.cstl	134	122	Exogenous	827	4
mathocl2mamba.cstl	134	166	Exogenous	644	3
restructure.cstl	173	173	Endogenous	127	1
Averages	142	154		908	8.2

Experiment Results

Table 9 summarises the size and development effort for the considered CGTL-coded transformations. Grammar size is given as the number of production rules in the grammar.

Table 10 shows the corresponding data for transformations coded using Java or UML-RSDS. Metamodel sizes are given as the sum of the number of metaclasses plus the total number of inheritances and (data) features in the metamodel. All work in Tables 9, 10 was performed by the first author using similar work practices.

Discussion

Table 9 shows that the average productivity for the CGTL-coded transformation scripts is about 111 LOC/person day, a similar result to the application of ANTLR/CGTL for DSL tools. This demonstrates that certain kinds of model transformation can be effectively developed using the approach.

For the Java/UML-RSDS-coded transformations of Table 10 the average productivity is 12.5 LOC/day or 274.5 LOC/month, a figure considerably below the CGTL transformation productivity. The effort involved in these transformations is also much higher, with an average of 300 person days for each transformation. *simplify.cstl* is the most complex of the CGTL-coded transformations, comparable in complexity and size to the ATL to UML-RSDS transformation. However, the development effort for *simplify.cstl* is only 25% of the ATL to UML-RSDS effort. Therefore CGTL appears to be a more appropriate transformation language for coding such a transformation.

Table 10 Size and development effort model transformations specified in Java or UML-RSDS

Transformation	Metamodel size		Transformation kind	Size (LOC)	Effort (person days)
	Source	Target			
UML2C [44]	158	87	Exogenous	6824	550
ATL2umlrsds [33]	85	158	Exogenous	1943	88
ETL2umlrsds [33]	22	158	Exogenous	1254	66
QVT2umlrsds [37]	62	158	Exogenous	5000	500
Averages	82	140		3755	301

Related Work

MDE usability issues have been highlighted by several studies of MDE in practice [1, 5, 12, 26, 58]. In response to these identified problems, different processes for using MDE have been proposed, such as the combination of agile methods with MDE [4]. Automated assistance for MDE processes, using AI techniques, has also been proposed [13]. One promising approach for improving the usability of MDE is *modelling by-example* [11, 38, 42]. Applied to DSL engineering, this concept includes the automated learning of natural language to DSL mappings [18], and applied to model transformation and code generator synthesis, it involves the derivation of transformation/generator rules from relatively small datasets of examples [38, 43]. In the case of DSL tooling, the example-based process has the drawback that there may be ambiguous results for learnt mappings, which must be resolved by the user [18].

Another approach for DSL design is the use of modelling assistants to construct the language design [56]. This is not an issue which we directly address, although the simplicity of the ANTLR/CGTL approach does facilitate experimentation with different versions of a DSL grammar and tool scripts for the DSL.

Large language models (LLMs) [62] have been investigated for MDE modelling assistance by [15], however they conclude that the present generation of LLMs do not have sufficient modelling knowledge to effectively assist in such MDE tasks at present.

Our program translation approach is related to reverse and re-engineering approaches which use a formal intermediate language, such as [10, 46]. However, we use UML/OCL as the intermediate representation, instead of a formal specification language. This has the advantage of being more widely understood by software practitioners, and more widely supported by tools.

Limitations

The pattern-matching mechanism of CGTL, used to match input parse tree terms against the LHS of CGTL script rules, is limited to the immediate subterms of the input term. The mechanism could be extended to match at arbitrary depths within a term tree, but this would result in increased execution time. Instead, auxiliary functions may be defined within a script to inspect child terms to an arbitrary depth and return information to rules processing parent terms.

CGTL does not support advanced MT facilities such as *incrementality* or *bidirectionality* [26]. As with mainstream MT languages, the specification of complex mappings in CGTL may require the use of calls to routines defined in a 3GL, which can hinder comprehensibility and maintenance.

With regard to DSL definition and processing, we focus attention on textual DSLs. However, graphical DSLs (such as MathOCL) can also be handled by providing a textual version of the DSL and a graphical-to-text conversion function. This however involves the simultaneous use of two notations for the same language. As discussed in Sect. Comparison with Other Concrete Syntax Transformation Languages, there are substantial tooling overheads for the support of transformation rules expressed in graphical concrete syntax. Currently our approach is geared to work with ANTLR, but in principle other parsing technologies such as JavaCC could be used to produce parse trees.

With regards to program abstraction and translation, modern programming languages such as Java have extensive libraries and hence in these cases it would require very high resources to model the semantics of the complete language together with its libraries. Users of our translation tools may extend the CGTL abstraction scripts as required to add semantics for specific program libraries. We have provided supporting UML/OCL libraries for the most widely-used program library facilities such as mathematical and string functions and file processing [39].

Conclusions and Future Work

We have defined a grammar-based MDE approach for software language processing, using concrete syntax specifications of transformations. We have shown that the approach can effectively implement typical MDE language processing tasks such as generating code from DSL models and

abstracting semantic information from source code. We have also shown that certain kinds of model transformation rule can be expressed in CGTL.

Based on our experience of developing large-scale language processing tools over the last 20 years, the ANTLR/CGTL approach significantly accelerates the development of such tools compared to MT-based and 3GL-based approaches, and substantially reduces the effort required. Thus we believe this approach is of potential value to the general MDE community in improving MDE usability in this domain.

An interesting area for future work is the combination of symbolic and non-symbolic machine learning (such as LLMs) to learn program and language translations. Non-symbolic ML could be more effective than symbolic ML in learning large-scale translations involving thousands of special cases (e.g., abstractions of Java or Python library operations).

Acknowledgements The support of UK EPSRC Seedcorn funding from MDENet for the Requirements Formalisation and MathOCL projects is acknowledged. The support of CLMS UK Ltd for the MathOCL project is also acknowledged. Shekoufeh Kolahdouz Rahimi contributed to the requirements formalisation work. Lyan Alwakeel and Ziwen Yuan contributed to the comparative evaluations.

Data availability All data and artefacts used in this paper are provided at the repository zenodo.org/records/10039613.

Declarations

Conflict of interest The authors confirm that there are no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Abrahao S, Bourdeleau F, Cheng B, Kokaly S, Paige R, Stoerrle H, Whittle J. User experience for MDE. In: MODELS 2017; 2017.
2. Agarwal M, Talamadupula K, Martinez F, Houde S, Muller M, Richards J, Ross SI, Weisz JD. Using document similarity methods to create parallel datasets for code translation. arXiv:2110.05423v1; 2022.
3. Ahmad W, Tushar M, Chakraborty S, Chang K-W. AVATAR: a parallel corpus for Java-Python program translation. arXiv:2108.11590v2; 2023.
4. Alfraihi H, Lano K. The integration of agile development and MDE: a systematic literature review. In: Modelsward 2017; 2017.
5. Alfraihi H, Lano K. Trends and insights into the use of model-driven engineering: a survey. In: SAM/MODELS; 2023.
6. Alwakeel L, Lano K, Alfraihi H. Towards integrating machine learning models into mobile apps using AppCraft. In: AgileMDE workshop, STAF 2023; 2023.
7. ANTLR. https://www.antlr.org, accessed 1.12.2023; 2023.
8. Besova G, Steenken D, Wehrheim H. Grammar-based model transformations: Definition, execution and quality properties. Computer Languages, Systems & Structures. 2015;43:116–38.
9. Black F, Scholes M. The pricing of options and corporate liabilities. J Polit Econ. 1973;8:637–54.
10. Bowen JP, Breuer P, Lano K. Formal specifications in software maintenance: from code to Z++ and back again. Inf Softw Technol. 1993;35(11–12):679–90.
11. Burgueno L, Cabot J, Gerard S. An LSTM-based neural network architecture for model transformations. In: MODELS '19, pages 294–299; 2019a.
12. Burgueno L, Cabot J, Gerard S. The future of model transformation languages: an open community discussion. JOT. 2019;18(3).
13. Burgueno L, Clariso R, Gerard S, Li S, Cabot J. An NLP-based architecture for the autocompletion of partial domain models. In: CAiSE 2021, pages 91–106. Springer; 2021.
14. Cabot J. Learning ANTLR – a software modeling perspective. https://modeling-languages.com/learning-antlr-software-modeling/, accessed Dec. 2023; 2018.
15. Camara J, Troya J, Burgueno L, Vallecillo A. On the assessment of generative AI in modeling tasks. SoSyM. 2023;22.
16. ClearPath Enterprise Servers. COBOL ANSI-85 programming reference manual; 2015.
17. Delp C, et al. Mos 2.0 – modeling the next revolutionary mission operations system. In: 2011 Aerospace Conference. IEEE; 2011.
18. Desai A, Gulwani S, Hingorani V, Jain N, Karkare A, Marron M, Sailesh R, Roy S. Program synthesis using natural language. In: ICSE. 2016;2016:345–56.
19. Eclipse. ATL user guide. eclipse.org; 2019.
20. Eclipse Agile UML project. projects.eclipse.org/projects/modeling.agileuml, accessed 8.1.2024; 2024.
21. Eliasson U, Heldal R, Lantz J, Berger C. Agile model-driven engineering in mechatronic systems-an industrial case study. In: Model-Driven Engineering Languages and Systems: 17th International Conference, MODELS 2014, Valencia, Spain, September 28–October 3, 2014. Proceedings 17, pages 433–449. Springer; 2014.
22. Epsilon project. Epsilon Generation Language, https://projects.eclipse.org/projects/modeling.epsilon. Accessed 26.3.2024; 2024.
23. Gorschek T, Tempero E, Angelis L. On the use of software design models in software development practice: an empirical investigation. J Syst Softw. 2014;95:176–93.
24. Gronmo R, Moller-Pedersen B, Olsen G. Comparison of three model transformation languages. In: ECMDA-FA 2009, pages 2–17. Springer; 2009.
25. Haughton H, Tehrani SY, Lano K. MathOCL: a domain-specific language for financial modelling. In: Agile MDE Workshop, STAF 2023; 2023.
26. Hoppner S, Haas Y, Tichy M, Juhnke K. Advantages and disadvantages of (dedicated) model transformation languages. Empirical Software Engineering. 2022;27(159).
27. Hutchinson J, Whittle J, Rouncefield M, Kristoffersen S. Empirical assessment of MDE in industry. In: Proceedings of the 33rd international conference on software engineering, pages 471–480; 2011.
28. Izquierdo JC, Molina JG. Extracting models from source code in software modernization. Software Systems Modelling. 2014;13:713–34.

29. Java2Python. https://github.com/natural/java2python; 2021.
30. Kernighan B, Ritchie D. The C Programming Language. 2nd ed. Prentice Hall; 1988.
31. Kusel A, Schonbock J, Wimmer M, Kappel G, Retschitzegger W, Schwinger W. Reuse in model-to-model transformation languages: are we there yet? SoSym. 2015;14:537–72.
32. Lachaux M-A, Roziere B, Chanussot L, Lample G. Unsupervised translation of programming languages. arXiv:2006.03511v3; 2020.
33. Lano K. Agile model-based development using UML-RSDS. CRC Press; 2017.
34. Lano K. Program translation using Model-driven engineering. In: ICSE 2022 Companion Proceedings, pages 362–363; 2022.
35. Lano K, Alfraihi H, Kolahdouz-Rahimi S, Sharbaf M, Haughton H. Comparative case studies in agile model-driven development. In: FlexMDE 2018, MODELS 2018; 2018.
36. Lano K, Haughton H, Yuan Z, Alfraihi H. Program abstraction and re-engineering: an Agile MDE approach. In: SAM/MODELS 2023; 2023.
37. Lano K, Kolahdouz-Rahimi S. Implementing QVT-R via semantic interpretation in UML-RSDS. SoSyM; 2021.
38. Lano K, Kolahdouz-Rahimi S, Fang S. Model Transformation Development using Automated Requirements Analysis, Metamodel Matching and Transformation By-Example. ACM TOSEM. 2021;31(2):1–71.
39. Lano K, Kolahdouz-Rahimi S, Jin K. OCL libraries for software specification and representation. In: OCL 2022, MODELS 2022 Companion Proceedings; 2022.
40. Lano K, Malik N. Mapping procedural patterns to object-oriented design patterns. Autom Softw Eng. 1999;6(3):265–89.
41. Lano K, Xue Q. Agile specification of code generators for model-driven engineering. In: 2020 15th International Conference on Software Engineering Advances (ICSEA), pages 9–15; 2020.
42. Lano K, Xue Q. Code generation by example. In: Proceedings of the 10th International Conference on Model-Driven Engineering and Software Development (MODELSWARD), pages 84–92; 2022.
43. Lano K, Xue Q. Code generation by example using symbolic machine learning. Springer Nature Computer Science; 2023.
44. Lano K, Yassipour-Tehrani S, Alfraihi H, Kolahdouz-Rahimi S. Translating from UML-RSDS OCL to ANSI C. In: OCL 2017, STAF 2017, pages 317–330; 2017.
45. Lattner C, Adve V. LLVM: A compilation framework for lifelong program analysis and transformation. In: CGO 2004, pages 75–86. IEEE; 2004.
46. Liu X, Yang H, Zedan H. Formal methods for the re-engineering of computing systems. In: Compsac '97; 1997.
47. Malyay A, et al. On ML-based program translation: perils and promises. arXiv:2302.10812v1; 2023.
48. Microsoft Corp. Office VBA Reference. https://learn.microsoft.com/en-us/office/vba/api/overview; 2022.
49. Mirachi S, et al. Applying agile methods to aircraft embedded software. Software Practice and Experience. 2017;47:1465–84.
50. OMG. Object Constraint Language (OCL) 2.4 Specification; 2014.
51. OMG. MOF2 Query/View/Transformation specification, v1.3; 2016.
52. Ozkaya M, Erata F. Understanding practitioners' challenges on software modeling: A survey. Journal of Computer Languages. 2020;58: 100963.
53. Parkin A. COBOL for Students. Edward Arnold; 1982.
54. Rahimi S, Lano K, Lin C. Requirement formalisation using natural language processing and machine learning: a systematic review. In: MODELSWARD; 2023.
55. Sanders J. https://www.techrepublic.com/article/jpmorgans-athena-has-35-million-lines-of-python-code-and-wont-be-updated-to-python-3-in-time; 2019.
56. Segura A, de Lara J, Wimmer M. Modelling assistants based on information reuse. SoSyM; 2023.
57. Umar M, Lano K. Advances in automated support for requirements engineering: a systematic literature review. Requirements Engineering, 2024;1–31.
58. Whittle J, Hutchinson J, Rouncefield M, Burden H, Heldal R. A taxonomy of tool-related issues affecting the adoption of MDE. Sosym. 2017;16:313–31.
59. Xtext. https://projects.eclipse.org/projects/modeling.tmf.xtext; 2022.
60. Zaki-Ismail A, Osama M, Abdelrazek M, Grundy J, Ibrahim A. RCM-extractor: an automated NLP-based approach for extracting a semi-formal representation model from natural language requirements. AUSE. 2022;29(1):1–33.
61. Zhao L, et al. Natural language processing for requirements engineering: a systematic mapping study. ACM Computing Surveys; 2020.
62. Zhao W, et al. A survey of large language models. arXiv:2303.18223v10; 2023.
63. Zucker J, D'Leeuwen M. Arbiter: a domain-specific language for ethical machine learning. In: AIES '20; 2020.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.